DATA ENGINEERING

# Delta Lake Visual Guide

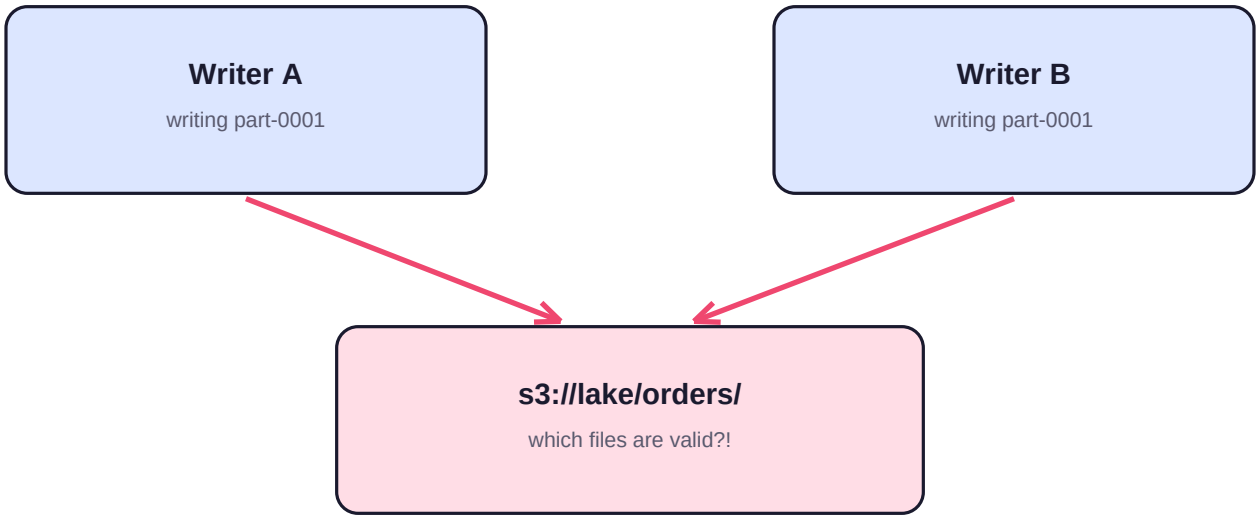
ACID transactions on your data lake.
Reliability without giving up cheap storage.

**ANIMATED WALKTHROUGH**
Step through pages to play the animation

STEP 1/6

# The Data Lake Reliability Problem

Parquet on object storage is cheap — but concurrent writes corrupt it.

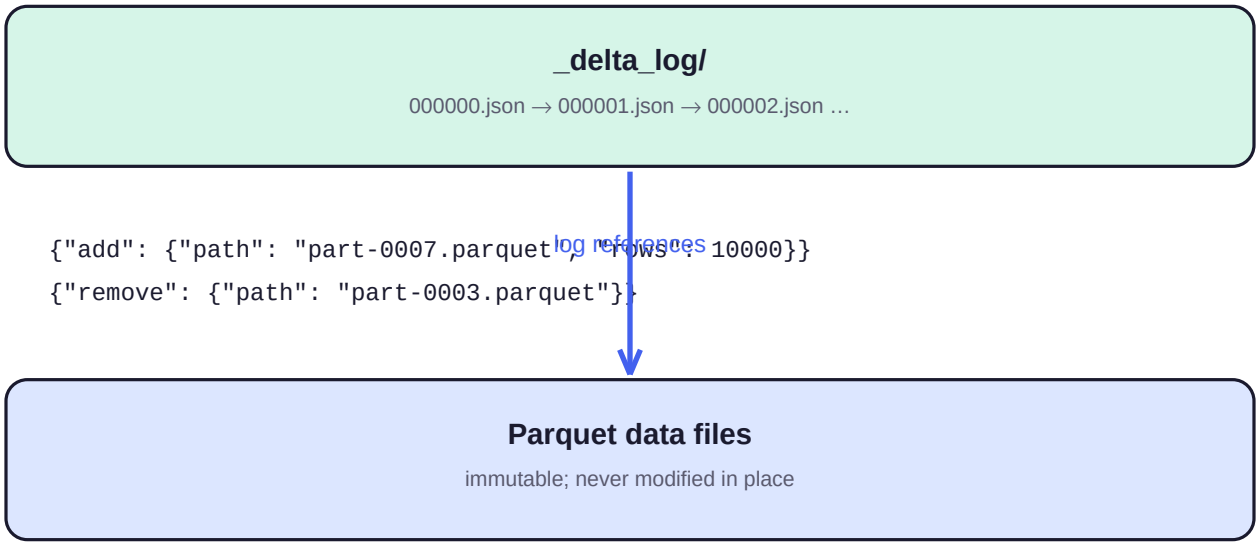



Two writers, no coordination: partial files, duplicate rows, readers seeing half-written data. Lakes need a transaction layer.

STEP 2/6

# The Transaction Log: The Source of Truth

Delta adds a _delta_log/ of ordered JSON commits on top of Parquet.




Writes are atomic: new Parquet files land first, then ONE log entry commits them. Readers replay the log to know exactly which files are current.

- Optimistic concurrency: conflicting writers retry automatically
- Checkpoints (every 10 commits) keep log replay fast
- The log is the table — files alone are meaningless

STEP 3/6

# ACID on Object Storage

Serializable isolation and atomic multi-file commits — on S3/ADLS/GCS.

**Atomicity**
all files or none

**Consistency**
schema enforced

**Isolation**
snapshot reads

Readers always see a consistent snapshot: a query starting at commit 42 never sees commit 43's partial state. Schema enforcement rejects bad writes; schema evolution allows deliberate, versioned changes.

This is what turns a 'data swamp' into a lakehouse: concurrent pipelines can safely read and write the same tables.

STEP 4/6

# Time Travel

Every commit is a version. Query any of them.

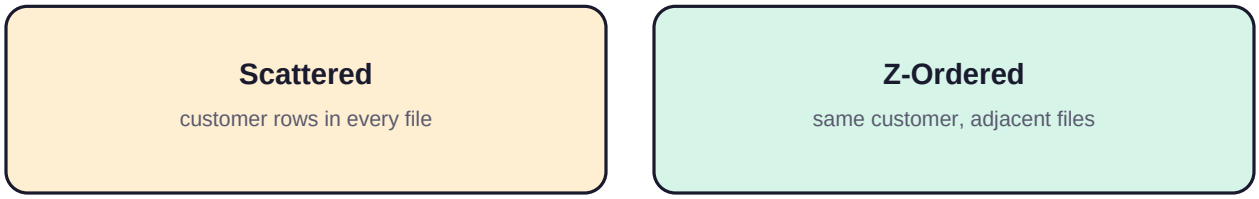
```
SELECT * FROM orders VERSION AS OF 1;
SELECT * FROM orders TIMESTAMP AS OF '2026-10-01';
```

Audits, reproducible ML training sets, and one-command rollback: RESTORE TABLE orders TO VERSION AS OF 1.

STEP 5/6

# Z-Ordering & OPTIMIZE

Co-locate related data so queries skip more files.




```
OPTIMIZE orders
  ZORDER BY (customer_id);
```

Compacts small files + multi-dimensional clustering.

Streaming writes create thousands of tiny files (the 'small file problem'). OPTIMIZE compacts them; ZORDER clusters by the columns you filter on.

STEP 6/6

# MERGE: Upserts Done Right

Atomic insert/update/delete in one statement — CDC's best friend.

```
MERGE INTO orders t USING updates s
  ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

One atomic commit applies the whole change set — no partial upserts, no torn reads. This is how CDC streams land cleanly into lakehouse tables.

- Change data feed: row-level change history for downstream consumers
- Deletes land as tombstones, then VACUUM reclaims old files
- Retention: keep 7+ days of history for time travel before vacuuming

STEP ✓/✓

# Key Takeaways

## Delta Lake — Remember

- The _delta_log/ transaction log turns Parquet files into an ACID table: atomic commits, snapshot isolation.
- Writers stage immutable Parquet files, then commit with a single log entry — readers replay the log.
- Time travel queries any version; RESTORE rolls back bad writes in one command.
- OPTIMIZE compacts small files; ZORDER BY co-locates related rows for faster skipping.
- MERGE gives atomic upserts — the clean landing pattern for CDC streams.

**Keep exploring**

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev